

Compiladores Aumentados con IA para Accesibilidad en Entornos de Programación

1^{er} Jhonatan B. Uscca Giraldo

Departamento de Ciencias de la Computación
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
juscag@unsa.edu.pe

2nd Leonardo Montoya Choque

Departamento de Ciencias de la Computación
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
lmontoyac@unsa.edu.pe

Abstract—Este trabajo presenta el diseño e implementación inicial de *SemanticC*, una herramienta de línea de comandos que actúa como asistente semántico accesible para código fuente, orientada a desarrolladores con discapacidad visual. A diferencia de asistentes de IA genéricos que operan sobre texto fuente crudo, *SemanticC* procesa el código a través de un compilador propio y usa las representaciones internas resultantes —tokens anotados, Árbol de Sintaxis Abstracta (AST), y en fases futuras tabla de símbolos y Grafo de Flujo de Control (CFG)— como contexto verificado para un modelo de lenguaje de gran escala (LLM). Esto permite responder consultas con respuestas ancladas en la semántica formal del programa, no en inferencias sobre texto ambiguo. Como sustrato de demostración se diseña *MiniLang*, un lenguaje imperativo de tipado estático implementado en Python. El compilador de *MiniLang* incluye un analizador léxico completo con 68 tipos de token, un analizador sintáctico por descenso recursivo con gramática EBNF completa y generación de AST con patrón Visitor, validado por 78 pruebas automatizadas.

Index Terms—Accesibilidad, Compiladores, Inteligencia Artificial, Discapacidad Visual, AST, Inclusión, LLM, Ingeniería de Software.

I. INTRODUCCIÓN

El acceso equitativo a herramientas de desarrollo de software continúa siendo un desafío significativo para desarrolladores con discapacidad visual. Aunque los entornos de desarrollo integrados (IDE) modernos han incorporado mecanismos básicos de accesibilidad, gran parte de la interacción con el código sigue dependiendo de representaciones visuales complejas, navegación espacial y retroalimentación gráfica [1].

Trabajos previos como CodeTalk [1] demostraron que los desarrolladores con discapacidad visual enfrentan barreras estructurales relacionadas con la *discoverability*, *glanceability*, *navigability* y *alertability*. Estas dificultades no surgen únicamente de la interfaz gráfica, sino también de la forma en que la información semántica del código es comunicada al usuario. En la mayoría de herramientas actuales, el compilador produce diagnósticos y representaciones internas altamente precisas —como Árboles de Sintaxis Abstracta (AST), tablas de símbolos y grafos de flujo de control (CFG)— que posteriormente son reducidas a mensajes lineales y visuales.

En paralelo, el crecimiento reciente de asistentes basados en modelos de lenguaje de gran escala (LLM) ha abierto nuevas posibilidades para la explicación automática de código y generación de lenguaje natural. Estudios recientes muestran

que estas herramientas pueden mejorar la productividad de desarrolladores con discapacidad visual [3]. Sin embargo, dichos sistemas suelen operar únicamente sobre texto fuente, sin integrar formalmente la semántica interna del compilador, lo que introduce problemas de ambigüedad, explicaciones inconsistentes y alucinaciones.

En este contexto, el presente trabajo propone el diseño e implementación de *SemanticC*, una herramienta CLI (*command-line interface*) que actúa como asistente semántico accesible para código fuente. *SemanticC* procesa código a través de un compilador propio —con fases léxica, sintáctica, semántica y de CFG— y expone las representaciones internas resultantes (tokens, AST, tabla de símbolos, CFG) a una capa de interpretación basada en un LLM. Esto permite que el modelo genere respuestas ancladas en la semántica formal del programa, no en texto fuente ambiguo.

A diferencia de un asistente de IA genérico, *SemanticC* responderá consultas específicas sin depender de interfaces visuales. El objetivo central es evaluar si el uso de representaciones semánticas del compilador como contexto para un LLM mejora la precisión, la comprensibilidad y la utilidad de las explicaciones para desarrolladores con discapacidad visual, en comparación con herramientas que operan sobre texto plano.

II. TRABAJOS RELACIONADOS

A. Accesibilidad en Entornos de Programación

Potluri et al. [1] presentaron CodeTalk, un sistema orientado a mejorar la accesibilidad en Visual Studio para desarrolladores con discapacidad visual. El trabajo identifica cuatro barreras fundamentales: *discoverability*, *glanceability*, *navigability* y *alertability*. Su propuesta incorpora navegación estructural y retroalimentación auditiva para facilitar la comprensión del código.

Posteriormente, herramientas como Accessible Blockly [2] exploraron representaciones jerárquicas del código basadas en Árboles de Sintaxis Abstracta (AST), permitiendo navegación estructural mediante teclado y lectores de pantalla. Estos trabajos demuestran que las representaciones estructurales del programa pueden reducir la dependencia de lectura línea por línea.

No obstante, la mayoría de soluciones existentes operan principalmente a nivel de interfaz o IDE, sin explotar directa-

mente las representaciones semánticas internas generadas por el compilador.

B. IA Generativa y Comprensión de Código

El auge reciente de modelos de lenguaje de gran escala (LLM) ha impulsado investigaciones sobre explicación automática de código y asistencia inteligente en programación. Estudios recientes [3] muestran que asistentes como GitHub Copilot pueden mejorar la productividad de desarrolladores con discapacidad visual, especialmente en tareas de generación y comprensión de código.

Sin embargo, estos sistemas presentan limitaciones importantes debido a que operan principalmente sobre texto fuente, sin restricciones semánticas formales provenientes del compilador. Como consecuencia, pueden producir explicaciones ambiguas o incorrectas, fenómeno conocido como *hallucination*.

C. Representaciones Semánticas en Compiladores

Los compiladores modernos generan múltiples representaciones estructurales del programa, incluyendo Árboles de Sintaxis Abstracta (AST), tablas de símbolos y Grafos de Flujo de Control (CFG). Estas estructuras permiten modelar formalmente la semántica del código y constituyen la base de procesos como análisis semántico, optimización y generación de diagnósticos.

A pesar de su riqueza semántica, estas representaciones rara vez son expuestas directamente como mecanismos de accesibilidad para el usuario final. Esta limitación evidencia una brecha entre la comprensión formal del programa por parte del compilador y la comprensión humana accesible del mismo.

D. Brecha Identificada

La literatura actual aborda accesibilidad principalmente desde la interfaz de usuario o desde asistentes basados en IA. Sin embargo, existe escasa investigación orientada a integrar simultáneamente:

- representaciones semánticas del compilador,
- mecanismos de accesibilidad estructural,
- y modelos de IA para explicación adaptativa.

Esta brecha motiva la presente investigación, que propone utilizar el compilador como fuente semántica confiable para generar explicaciones accesibles asistidas por IA.

III. DISEÑO DE SEMANTICC

SemanticC es una herramienta CLI que recibe un archivo de código fuente MiniLang y una consulta del usuario, y devuelve una respuesta en lenguaje natural basada en las representaciones semánticas internas del compilador. No produce código ejecutable; su salida es semántica accesible. La Figura 1 muestra el pipeline completo.

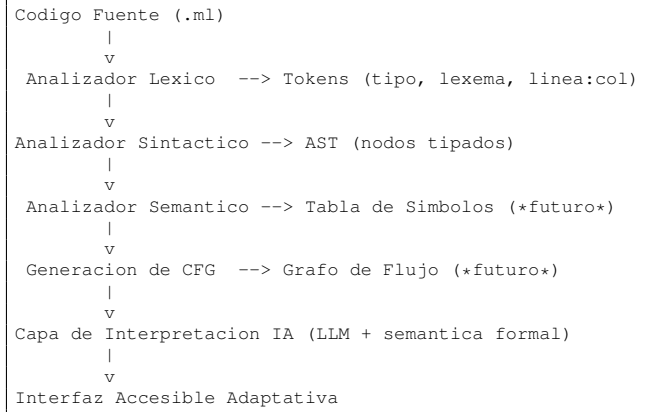


Fig. 1. Pipeline de SemanticC. Las fases marcadas como “futuro” están planificadas pero aún no implementadas. Las fases implementadas actualmente son Léxico y Sintáctico.

A. Capa Semántica del Compilador

Esta capa es el núcleo técnico de la propuesta. A diferencia de sistemas que operan sobre texto plano, el compilador genera representaciones formales y verificadas del programa:

- **Tokens anotados:** cada unidad léxica incluye tipo, lexema, valor literal y posición exacta (línea:columna), lo que permite diagnósticos precisos.
- **AST tipado:** el árbol de sintaxis abstracta refleja la estructura jerárquica del programa, exponiendo relaciones entre funciones, bloques y expresiones que son invisibles en la lectura lineal.
- **Tabla de símbolos** (fase futura): registro de variables y funciones con tipo, alcance y sitio de declaración.
- **CFG** (fase futura): grafo de flujo de control que modela caminos de ejecución, bucles y ramas condicionales.

B. Capa de Interpretación Basada en IA

Esta capa recibe como entrada las estructuras generadas por el compilador (no el texto fuente crudo) y construye un *prompt* estructurado que el LLM usa para formular su respuesta. Al operar sobre representaciones formalmente correctas, el modelo no puede “alucinar” la estructura del programa: si el AST dice que `factorial` es una función recursiva que retorna `int`, el LLM trabaja desde esa certeza.

IV. IMPLEMENTACIÓN: COMPILADOR DE MINILANG

Para demostrar la arquitectura propuesta de forma controlada y reproducible se diseña *MiniLang*, un lenguaje imperativo de tipado estático implementado en Python 3.14. La elección de un lenguaje propio permite controlar completamente las estructuras internas expuestas a la capa de IA y producir diagnósticos pedagógicamente ricos.

A. Características de MiniLang

MiniLang soporta los siguientes constructos:

- Tipos primitivos: `int`, `float`, `bool`, `string`.
- Declaración de variables tipadas: `int x = 5;`.

- Funciones con parámetros y tipo de retorno: `func int f(int a) { }`.
- Estructuras de control: `if/else`, `while`, `for`, `break`, `continue`.
- Instrucciones de I/O integradas: `print(...)`, `read(...)`.
- Operadores aritméticos, relacionales, lógicos y de asignación compuesta.
- Comentarios de línea (`//`) y de bloque (`/* */`).

El fragmento de código a continuación muestra un programa de ejemplo que calcula el factorial de forma recursiva:

```
1 func int factorial(int n) {
2     if (n <= 1) {
3         return 1;
4     }
5     return n * factorial(n - 1);
6 }
7
8 func void main() {
9     int numero = 10;
10    int resultado = factorial(numero);
11    print("factorial de ", numero, " es: ",
12         resultado);
13 }
```

Listing 1. factorial.ml – programa de prueba en MiniLang

B. Analizador Léxico

El analizador léxico (*Lexer*) transforma el texto fuente en una secuencia de *tokens*, cada uno anotado con tipo, lexema, valor literal y posición exacta en la fuente. La Tabla I resume las categorías de tokens reconocidas.

TABLE I
CATEGORÍAS DE TOKENS DE MINILANG

Categoría	Ejemplos
Literales	42, 3.14, "hola", true
Palabras reservadas	int, func, if, while, ...
Operadores aritméticos	+, -, *, /, %
Operadores relacionales	==, !=, <, <=, >, >=
Operadores lógicos	&&, , !
Asignación compuesta	=, +=, -=, *=, /=
Delimitadores	(,), {, }, ;, ,

El lexer implementa reporte de errores con posición exacta (línea:columna), soporte a secuencias de escape en cadenas y comentarios de línea y bloque. Un fragmento del flujo de tokens producido para el programa `hello.ml` ilustra la información disponible:

```
Token(KW_FUNC, 'func', lit=None, 4:1)
Token(KW_VOID, 'void', lit=None, 4:6)
Token(IDENTIFIER, 'main', lit=None, 4:11)
Token(KW_STRING, 'string', lit=None, 5:5)
Token(IDENTIFIER, 'nombre', lit=None, 5:12)
Token(EQ, '=', lit=None, 5:19)
Token(STRING, '"Mundo"', lit='Mundo', 5:21)
Token(KW_PRINT, 'print', lit=None, 6:5)
```

Listing 2. Flujo de tokens anotado de hello.ml

Esta riqueza léxica es la que permite a la capa de IA referirse a posiciones precisas del código al generar explicaciones o diagnósticos accesibles.

C. Analizador Sintáctico y AST

El analizador sintáctico implementa un parser de *descenso recursivo predictivo* basado en la gramática EBNF de MiniLang. El parser produce un AST compuesto por más de 20 tipos de nodo, organizados en la jerarquía mostrada en la Tabla II.

TABLE II
JERARQUÍA DE NODOS DEL AST DE MINILANG

Categoría	Nodos
Programa	Program, Block
Declaraciones	FunctionDecl, VarDecl
Control	IfStmt, WhileStmt, ForStmt
Sentencias	ReturnStmt, BreakStmt, ContinueStmt
I/O	PrintStmt, ReadStmt
Expresiones	BinaryOp, UnaryOp, AssignExpr
Llamadas	CallExpr, IdentifierExpr
Literales	IntLiteral, FloatLiteral, StringLiteral, ...

Los nodos implementan el patrón *Visitor*, lo que permite añadir nuevos recorridos del AST (impresión, análisis semántico, generación de CFG, serialización para la capa de IA) sin modificar las clases de nodo. La Figura ?? muestra el AST generado para el programa `factorial.ml`:

```
Program
  FunctionDecl name=factorial return=int params=(
    int n)
  Block
    IfStmt
      condition:
        BinaryOp op='<='
        Identifier name=n
        IntLiteral 1
      then:
        Block
          ReturnStmt
            IntLiteral 1
        ReturnStmt
          BinaryOp op='*'
          Identifier name=n
          CallExpr name=factorial args=1
            BinaryOp op='-'
            Identifier name=n
            IntLiteral 1
  FunctionDecl name=main return=void params=()
  Block
    VarDecl type=int name=numero
    VarDecl type=int name=resultado
    PrintStmt (4 arg(s))
```

Listing 3. AST generado para factorial.ml por el ASTPrinter

Esta representación estructurada es precisamente la que puede ser consultada por un LLM para generar descripciones accesibles. Por ejemplo, dado el nodo `IfStmt` con su condición `BinaryOp op='<='`, el sistema puede generar explicaciones sin ambigüedad semántica.

D. Manejo de Errores

El compilador genera errores léxicos y sintácticos con posición exacta (línea y columna), lo que constituye la base para diagnósticos accesibles. Ejemplos de mensajes de error:

```
[LexerError] Linea 3, Col 8: Cadena de texto sin
cerrar
```

```
[ParseError] Linea 7, Col 1: Se esperaba ';' al
final de la declaracion
[ParseError] Linea 12, Col 5: Se esperaba ')' al
cerrar condicion
```

V. VALIDACIÓN EXPERIMENTAL PRELIMINAR

La implementación actual fue validada mediante una suite de 78 pruebas automatizadas desarrolladas con `pytest`. Las pruebas cubren las siguientes dimensiones:

- **Léxico** (31 pruebas): literales de todos los tipos, palabras reservadas, operadores simples y compuestos, delimitadores, comentarios, seguimiento de línea y columna, y detección de 5 categorías de error léxico.
- **Sintáctico** (47 pruebas): declaraciones de variables y funciones, todas las estructuras de control, sentencias de I/O, 12 formas de expresión con precedencia correcta, y 5 categorías de error sintáctico. Se incluye un test de integración con el programa `factorial.ml` completo.

Los 78 tests pasan satisfactoriamente, confirmando la corrección del analizador léxico y sintáctico de MiniLang.

VI. TRABAJO FUTURO

Las siguientes fases de desarrollo están planificadas:

- 1) **Analizador semántico**: verificación de tipos, resolución de alcances y construcción de la tabla de símbolos.
- 2) **Generación de CFG**: construcción del grafo de flujo de control a partir del AST, para exponer la estructura de ejecución del programa.
- 3) **Integración con LLM**: diseño del prompt estructurado que combina información del AST, tabla de símbolos y CFG como contexto para el modelo de lenguaje.
- 4) **Interfaz accesible**: desarrollo de la capa de salida orientada a lectores de pantalla y navegación por teclado.
- 5) **Evaluación con usuarios**: estudio empírico con desarrolladores con discapacidad visual, midiendo comprensión, navegación y depuración de código usando métricas como NASA-TLX y tiempo de resolución de tareas.

REFERENCES

- [1] V. Potluri, A. Begel, M. Barnett, et al., "CodeTalk: Improving Programming Environment Accessibility for Visually Impaired Developers," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, ACM, 2018.
- [2] E. Schanzer and S. Bahram, "Accessible Blockly: An Accessible Programming Blocks Editor," in *Proceedings of ACM SIGACCESS*, 2022.
- [3] S. Savage, C. Flores-Saviaga, et al., "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired," arXiv preprint arXiv:2503.16491, 2025.